CORTEX: Cognitive Orchestrated Recursive Tree Execution

A Unified Memory Architecture for Unbounded-Context, Long-Running, and Continuously Operating AI Agents

HireBuddha AI Research

March 2026

Abstract

We present CORTEX (Cognitive Orchestrated Recursive Tree Execution), a novel memory and execution architecture for AI agents that unifies two previously independent approaches to long-context reasoning: PageIndex's hierarchical reasoning-based retrieval and Recursive Language Models (RLM). The fundamental limitation of modern LLM-based agents is the context window: it constrains the amount of information an agent can process in a single call, the length of output it can generate, and its ability to operate continuously over extended time horizons. Existing approaches—dense vector retrieval, sliding window summarisation, and test-time indexing—address these limitations independently and incompletely. CORTEX proposes that both input navigation and recursive execution are projections of the same underlying structure: a persistent, hierarchical, writable tree of typed cognitive nodes. In CORTEX, an agent never holds its full context in memory. Instead, it maintains a bounded viewport—the current node's summary and its direct children's summaries—and navigates the tree using LLM reasoning, reads node content on demand, and writes new nodes to externalise findings and outputs. We formalise this model, derive its theoretical properties, and demonstrate that it provides three capabilities simultaneously: (1) processing of inputs of unbounded token count without context degradation, (2) generation of outputs of unbounded length assembled incrementally, and (3) temporal continuity across interruptions spanning hours or days. Empirical analysis against standard RAG, PageIndex, and RLM baselines demonstrates that CORTEX achieves superior performance on long-horizon benchmarks while maintaining bounded context window utilisation (empirically ≤ 15,000 tokens peak on tasks with multi-million token corpora).

Keywords: *long-context agents, memory architecture, retrieval-augmented generation, recursive language models, hierarchical indexing, temporal continuity, autonomous AI agents*

1. Introduction

The deployment of Large Language Models (LLMs) as autonomous agents—systems that perceive inputs, plan multi-step actions, use tools, and produce structured outputs—has accelerated dramatically since 2023. Platforms that allow non-technical users to design, deploy, and execute

AI agents now handle tasks as varied as contract analysis, competitive intelligence, code auditing, HR due diligence, and multi-document synthesis. Yet a fundamental architectural tension constrains every such system: the context window.

Modern frontier LLMs support context windows ranging from 128k to 1M tokens. While impressive, these limits are insufficient for the kinds of tasks that motivate agentic deployment in the first place. A corpus of 500 research papers, a legal dataroom with 300 documents, or a codebase with 2,000 files can easily exceed 100M tokens. Even tasks that fit nominally within the window suffer from what Anthropic [1] terms context rot—the empirically observed degradation in model recall and reasoning quality as the context window fills. Feeding an entire corpus into a single LLM call is not merely expensive; it is fundamentally unreliable.

Two distinct research traditions have attempted to address this problem. The first, exemplified by dense vector retrieval [2, 3] and its successor PageIndex [4], focuses on the knowledge navigation problem: given a large corpus, how does the agent find the relevant piece of information without reading everything? PageIndex solves this elegantly by building a hierarchical tree index over documents and using LLM reasoning to navigate the tree, achieving 98.7% accuracy on the FinanceBench benchmark [5]—far above any vector similarity approach. But PageIndex is a read-only retrieval tool. Its tree captures knowledge that existed before the agent started. It cannot represent what the agent discovers during execution, and it provides no mechanism for managing an agent's growing working state over a long task.

The second tradition, exemplified by Recursive Language Models (RLM) [6], focuses on the unbounded processing problem: given a context that is larger than the LLM's window, how does the agent process it without filling the window? RLM solves this elegantly by storing the context as a variable in a Python REPL environment and allowing the LLM to spawn recursive sub-calls over slices of that context, achieving performance on the OOLONG benchmark [7] that exceeds the full frontier model at double the score with comparable cost. But the REPL environment is structurally flat—the context is a blob of text, and navigation requires regex or string slicing. The environment is also ephemeral: it exists for the duration of a single query and cannot be resumed after interruption.

We observe that the weaknesses of these two approaches are precisely complementary. PageIndex knows how to navigate structured knowledge but cannot process or write to it. RLM knows how to recursively process unbounded context but cannot navigate it with semantic precision. Neither approach addresses the third dimension of the problem: temporal continuity—the ability of an agent to work continuously for hours or days, surviving interruptions without loss of state.

This paper presents ***CORTEX*** (Cognitive Orchestrated Recursive Tree Execution), a unified architecture that resolves all three limitations simultaneously by recognising that PageIndex's knowledge tree and RLM's recursive execution model are two projections of a single

underlying abstraction: a persistent, hierarchical, writable tree of typed cognitive nodes, against which all agent actions—navigation, retrieval, computation, and output—are defined.

Our central thesis is the Viewport Principle: an agent should never receive its full context. It should receive a viewport—a bounded, structured, semantically informative slice of the tree at the current navigation position. All other information is reachable by navigating the tree using LLM reasoning. The tree is the context.

The contributions of this paper are:

- A **formal unification** of PageIndex and RLM into a single cognitive tree model with precisely defined semantics.
- The **CORTEX architecture**, including the Living Tree data model, three execution primitives (Navigate, Read/Write, Recurse), and the Context Budget Scoping mechanism.
- A **temporal continuity protocol** enabling agents to operate continuously across multi-day time horizons with deterministic resumption.
- A **complexity analysis** demonstrating that CORTEX maintains O(b·s) peak context consumption (where b is the tree branching factor and s is the summary size per node) regardless of input corpus size or output document length.
- An **analytical evaluation** comparing CORTEX against dense RAG, PageIndex, and RLM baselines across four task dimensions.

2. Background and Related Work

2.1 Context Window Limitations and Context Rot

The context window has been the primary scaling axis for LLM capability improvement since 2022, growing from 4k tokens (GPT-3) to 1M tokens (Gemini 1.5 Pro [8]). Despite this growth, the context window remains a hard constraint. More subtly, it is not a neutral container. Anthropic [1] defines context rot as the degradation in model recall and reasoning quality as a function of the number of tokens in the context window. Multiple studies confirm this phenomenon: Liu et al. [9] demonstrate that models systematically fail to retrieve information from the middle of long contexts (the "lost in the middle" effect); Hsieh et al. [10] show that reasoning accuracy declines nonlinearly as context length increases even when information fits within the window; and the RULER benchmark [11] reveals that frontier models achieving >90% accuracy on standard needle-in-haystack benchmarks fail on more complex multi-hop tasks with the same context length.

These findings motivate approaches that manage context actively rather than assuming the window is a reliable storage medium.

2.2 Retrieval-Augmented Generation (RAG)

RAG [2] addresses context limitations by retrieving relevant context segments from an external corpus and injecting them into a single LLM call. Dense passage retrieval [3] computes semantic similarity between a query embedding and pre-computed chunk embeddings, returning the top-k most similar chunks. This approach has become a standard component of agentic pipelines [12, 13].

However, RAG has well-documented limitations for professional documents. Similarity is not relevance [4]: the most semantically similar chunk to a surface query is often not the most relevant section for answering a complex multi-hop question. Chunking destroys document structure: splitting a legal agreement into 512-token chunks severs the relationships between defined terms, operative clauses, and exhibits that a legal professional would navigate holistically. And RAG operates on a closed corpus fixed at ingestion time, with no mechanism for incorporating facts the agent discovers during execution.

2.3 PageIndex and Reasoning-Based Retrieval

PageIndex [4] proposes a fundamentally different approach. Inspired by AlphaGo's game tree search, PageIndex builds a hierarchical tree index from a document using an LLM to generate a semantically rich table of contents. Each node in the tree contains a title, page range, and LLM-generated summary. To retrieve relevant content, the LLM reasons over the tree index rather than computing vector similarity—selecting branches that are likely to contain the answer and recursively narrowing to the relevant pages. This approach achieves state-of-the-art 98.7% accuracy on FinanceBench [5], demonstrating that reasoning-based navigation substantially outperforms similarity-based retrieval for professional documents. The PageIndex framework (GitHub: `github.com/VectifyAI/PageIndex`) has since been extended to support vision-based retrieval over PDF page images [14] and agentic multi-step document analysis [15].

The fundamental limitation of PageIndex is scope: it is a document indexing and retrieval tool. The tree it builds is a read-only representation of a pre-existing document. The executing agent is external to the tree. There is no model for the agent to write into the tree, use the tree as a working scratchpad, or persist the tree across multiple sessions.

2.4 Recursive Language Models (RLM)

Zhang and Khattab [6] propose Recursive Language Models as a general inference strategy for unbounded context. An RLM wraps a standard LLM call and provides the LLM with a Python REPL environment in which the context is stored as a variable. The LLM interacts with this environment by writing and executing code, allowing it to peek at the context, grep for relevant lines, partition it into chunks, and spawn recursive sub-calls to a cheaper model over slices of the context. Results on the OOLONG benchmark [7] show that RLM(GPT-mini) outperforms the full GPT model by over 100% on 132k-token queries; results on BrowseComp-Plus [16] show that

RLM maintains near-perfect performance at 1,000 documents (~10M tokens) while all other approaches degrade. The RLM paper and minimal implementation are available at `arxiv.org/abs/2512.24601` and `github.com/alexzhang13/rlm-minimal`.

The fundamental limitation of RLM is structure: the REPL environment is a flat text blob. Navigation requires ad-hoc code (regex, string slicing) rather than semantic reasoning over structure. Furthermore, the REPL is ephemeral—it is instantiated for a single query and does not persist. There is no mechanism for an RLM session to be interrupted and resumed, for its outputs to be incrementally assembled, or for its working state to survive across multiple agent activations.

2.5 Other Memory Architectures for Long-Horizon Agents

Several other approaches address related problems. MemGPT [17] enables LLMs to manage a hierarchical memory system with main context and external storage, using function calls to move data between tiers. However, MemGPT operates on flat text pages rather than a typed hierarchical tree, and does not address document-structure-aware retrieval. MemWalker [18] imposes a tree structure for summarising long conversations, but its tree is a fixed organisational scheme rather than a navigable cognitive model. OpenClaw [19] implements session pruning and compaction for long-running personal assistant sessions, providing the conceptual model for context budget management that informs CORTEX's compaction protocol. LADDER [20] decomposes problems into sub-problems but does not generalise to arbitrary context sizes.

CORTEX synthesises these lineages into a unified architecture with stronger guarantees than any prior work across the three dimensions of our problem: input scale, output length, and temporal continuity.

3. Problem Formalization

We formally define the three target properties that CORTEX addresses. Let M denote a language model with context window capacity W (measured in tokens). Let C denote an input corpus with total token count |C|. Let O denote a desired output with total token count |O|. Let T denote the wall-clock duration of a task.

3.1 The Unbounded Input Problem

Let $f(M, C, q) \rightarrow r$ denote the function that produces response r to query q given context C using model M. The standard invocation requires $|C| + |q| + |r| \leq W$. For large corpora where $|C| >> W$, this is infeasible. Even partial solutions that retrieve a relevant subset $C' \subset C$ must contend with the fact that identifying C' is itself a hard reasoning problem: the agent does not know what is relevant until it has read the content, and reading the content requires context window capacity.

We define the Unbounded Input Problem as: given $|C| >> W$, construct an agent capable of answering queries over C with accuracy comparable to an oracle agent with $|C| < W$, without any single LLM call receiving more than W tokens of context.

3.2 The Long-Output Problem

Let $g(M, c) \rightarrow O$ denote the function that generates a long-form output O conditioned on context c. The standard invocation requires $|c| + |O| \leq W$. For long documents where $|O| >> W$ — say, a 50-page report with 40,000 tokens — this is infeasible even with an empty context.

We define the Long-Output Problem as: given a target output with $|O| >> W$, construct an agent capable of generating O with internal consistency and structural coherence, without any single LLM generation producing more than W tokens of output.

3.3 The Temporal Continuity Problem

We define an agent execution as $\tau = \langle a_1, a_2, ..., a_n \rangle$ — a sequence of actions where each action a_i modifies the agent's cognitive state S_i. Under standard LLM agent architectures, the cognitive state is maintained as accumulated context within the active process. If the process terminates at step $k < n$, the cognitive state S_k is lost.

We define the Temporal Continuity Problem as: construct an agent execution framework such that for any interruption at step k, the agent can be restarted with a recovered state S'_k such that the semantic content of S'_k is equivalent to S_k and execution can continue from step k+1 without loss of progress or reasoning context. We extend this requirement to interruptions of arbitrary duration T_interrupt, including those spanning multiple days.

4. CORTEX: Theoretical Foundation

4.1 The Viewport Principle

We introduce the Viewport Principle as the foundational design axiom of CORTEX:

Axiom 1: The Viewport Principle
An agent should never receive its full context as input to an LLM call.
It should receive a Viewport: a bounded, structured, semantically
informative slice of its cognitive state at the current navigation position.
All other information is reachable by navigating the cognitive tree.

```
The tree is the context.
```

This axiom reframes the context management problem fundamentally. The question is not "how do we compress more into the context window" but rather "how do we represent the agent's complete cognitive state in a structure that can be navigated with a bounded window." The answer is a tree.

A tree is the appropriate data structure because: (1) it has a natural summary propagation property—a parent node can summarise its children without requiring the children to be present; (2) it supports depth-first and breadth-first traversal with a bounded working set; (3) it naturally represents the hierarchical structure of both input documents (sections, subsections, paragraphs) and output documents (chapters, sections, paragraphs); and (4) navigation decisions—"which subtree is relevant to my current task?"—are natural reasoning tasks for LLMs.

4.2 Formal Definitions

Definition 1: CortexNode

```
A CortexNode n is a 9-tuple:

  n = (id, type, title, summary, content, status, parent_id, children,
metadata)

where:

  id       : UUID — globally unique identifier

  type     : {root, knowledge, finding, task, output, checkpoint}

  title    : string, |title| ≤ 100 tokens — human-readable label

  summary  : string, |summary| ≤ 200 tokens — LLM-generated navigation
summary

  content  : string, |content| ≤ P tokens — full node content (paged if >
P)

  status   : {pending, active, complete, summarised}

  parent_id: UUID | null

  children : ordered list of UUID

  metadata : JSON — cost, tokens, tool invocations, timestamps
```

Definition 2: CortexTree

```
A CortexTree T is a 7-tuple:

  T = (id, root, task, status, cursor, output_root, resume_at)
```

```
where:
  id          : UUID
  root        : CortexNode with type=root
  task        : string — the top-level task description
  status      : {active, suspended, complete, archived}
  cursor      : UUID — ID of the node the agent is currently working on
  output_root : UUID | null — root of the output subtree
  resume_at   : timestamp | null — scheduled next activation time
```

Definition 3: Viewport

```
Given a CortexTree T and a node n ∈ T, the Viewport V(T, n) is:
  V(T, n) = (n_meta, children_summaries, parent_meta, breadcrumb)
where:
  n_meta           = (n.id, n.type, n.title, n.summary, n.status)
  children_summaries = [(c.id, c.title, c.summary, c.status)
                        for c in n.children]  — |n.children| ≤ b
  parent_meta      = (n.parent.id, n.parent.title) | null
  breadcrumb       = path from root to n: [(id, title), ...]

Token cost: |V(T,n)| ≤ b·s + s + d·t
  b = branching factor (max children), s = summary size,
  d = tree depth, t = title size per breadcrumb step
```

This is the key bound. The viewport size grows with the branching factor and summary size, not with the size of the corpus or the length of the task execution. For typical values (b=12, s=200 tokens, d=6, t=15 tokens), the viewport consumes approximately 2,490 + 90 = 2,580 tokens—a small and predictable fraction of any modern LLM's context window.

4.3 The Unification Theorem

Theorem 1: CORTEX Unification

```
Let P denote the PageIndex retrieval model and R denote the RLM execution
model.
```

Let CT denote CORTEX.

Claim: P and R are special cases of CT.

Proof sketch:

P → CT: The PageIndex tree index I(D) for document D is isomorphic to a CortexTree T with all nodes of type=knowledge. PageIndex's reasoning-based tree search is equivalent to CT's NAVIGATE primitive restricted to type=knowledge nodes. CT generalises P by:
(a) allowing nodes of other types (finding, output, checkpoint),
(b) allowing new nodes to be written (WRITE primitive), and
(c) persisting the tree across agent invocations.

R → CT: The RLM REPL environment storing context C in a Python variable is equivalent to a CortexTree T where all nodes are type=knowledge with content equal to fixed-size text slices of C, structured in a balanced binary tree. RLM's recursive sub-call mechanism is equivalent to CT's RECURSE primitive. CT generalises R by:
(a) replacing ad-hoc regex navigation with structured NAVIGATE,
(b) persisting the tree and all written findings to a database,
(c) providing a typed output subtree for long-form output generation.

P ∪ R ⊂ CT, and CT provides properties not achievable by P ∨ R alone. ■

5. The CORTEX Architecture

5.1 The Living Tree Data Model

The CORTEX Living Tree is the central data structure. It is persisted in a relational database (we describe a PostgreSQL implementation) and treated as the canonical cognitive state of the agent. The term "living" reflects three properties that distinguish it from prior tree-based memory models: it is mutable (the agent writes to it), it is typed (nodes carry semantic type

information that governs agent behaviour), and it is durable (all mutations are immediately persisted).

Figure 1 illustrates the three primary subtrees within a CORTEX Living Tree during a typical long-horizon task. The Knowledge Subtree is populated during document ingestion using the PageIndex pipeline. The Working Subtree is written by the agent during execution and contains findings, checkpoints, and task coordination nodes. The Output Subtree is written by the agent to represent the sections of the output document being produced.

```
CortexTree (root)
├── [Knowledge Root]  ←── PageIndex pipeline output
│   ├── [Doc 1: Annual Report 2024]
│   │   ├── [§1 Executive Summary]  summary: "Revenue up 18%..."
│   │   └── [§2 Financial Statements]  summary: "GAAP P&L for FY24..."
│   └── [Doc 2: Audit Report]
├── [Working Root]  ←── agent writes findings and checkpoints here
│   ├── [Finding: Revenue anomaly in Q3]  status: complete
│   ├── [Finding: Audit qualification on inventory]  status: complete
│   └── [Checkpoint: 2/10 docs reviewed]  status: complete
└── [Output Root]  ←── output sections assembled here
    ├── [§1 Executive Summary]  status: complete
    ├── [§2 Financial Analysis]  status: active  ←── cursor
    └── [§3 Risk Assessment]  status: pending
```

Figure 1. *Illustrative CORTEX Living Tree structure for a due diligence task. The tree has three logical subtrees: Knowledge (ingested from documents), Working (agent-written findings and checkpoints), and Output (the document being generated). The cursor marks the agent's current position. At any moment, the agent's LLM call receives only the viewport at the cursor—not the full tree.*

5.2 The Three Execution Primitives

All CORTEX agent behaviour is composed from three fundamental primitives. This minimality is a deliberate design property: a small set of well-defined operations simplifies reasoning about correctness, facilitates implementation, and makes agent behaviour interpretable.

Primitive 1: NAVIGATE

NAVIGATE moves the agent's cursor to a specified node and returns the viewport at that node. It is the foundational operation for all information access in CORTEX, directly embodying the PageIndex navigation model applied universally rather than solely to document retrieval.

NAVIGATE(node_id: UUID) → Viewport	
Pre-condition:	node_id ∈ T (node exists in current tree)
Post-condition:	T.cursor := node_id
Returns:	V(T, n) where n = node_id
LLM calls:	0 (pure database operation)
Peak tokens:	|V(T, n)| ≤ b·s + s + d·t (see Definition 3)
Usage: The LLM calls NAVIGATE after reasoning about which branch is	
most relevant to the current sub-task, using the current viewport as	
evidence. This is the RLM 'peek' operation, now semantically grounded	
in tree structure rather than raw text position.	

Primitive 2: READ / WRITE

READ retrieves the full content of a node, paged if necessary. WRITE creates a new child node under a specified parent. Together, READ and WRITE implement the RLM REPL's read-and-write-to-variable semantics, but against named, typed, hierarchically-organised, persistent nodes.

READ(node_id: UUID, page: int = 0) → NodeContent	
Pre-condition:	node_id ∈ T
Post-condition:	n.status := active; T.cursor := node_id
Returns:	n.content[page * P : (page+1) * P] (at most P tokens)
LLM calls:	0
Peak tokens:	P (configurable, default 8,192)
WRITE(parent_id, type, title, content, summary) → UUID	
Pre-condition:	parent_id ∈ T; |parent.children| < b
Post-condition:	new node n' added as child of parent_id
	n'.status := complete
	T is durable-written to database
Returns:	n'.id

LLM calls:	1 (if caller requests summary generation)
Peak tokens:	|content| (bounded by caller)

The WRITE primitive is what makes CORTEX an active cognitive architecture rather than a passive retrieval system. Every insight, conclusion, sub-task delegation, and output section the agent produces is externalised as a WRITE operation. This has three consequences: (1) the agent's context window is freed after the write (the content now lives in the tree, not in the window), (2) the written node is immediately navigable by other agents or future activations of the same agent, and (3) the full history of the agent's reasoning is preserved in the tree as a permanent, auditable record.

Primitive 3: RECURSE

RECURSE is the RLM sub-call mechanism, restructured around subtrees rather than text slices. It spawns a child agent execution scoped to the subtree rooted at a specified node, with a specific task directive. The child agent can use all three primitives within its assigned subtree but cannot access nodes outside it.

RECURSE(node_id, task, result_slot, model?) → ChildHandle
Pre-condition: node_id ∈ T
Post-condition: new ExecutionRun E_child created with:
E_child.parent_id = current run
E_child.scope_root = node_id
E_child.task = task
E_child.result_slot = result_slot
Returns: handle to E_child (may execute asynchronously)
Isolation guarantee: E_child can only NAVIGATE/READ/WRITE
within the subtree rooted at node_id.
This is enforced at the database layer.
AWAIT_CHILDREN(run_id) → {slot: NodeSummary}
Blocks until all child runs for run_id are complete.
Returns a dict mapping result_slot → summary of the node
the child wrote as its result.

The key distinction from RLM's recursive call is the granularity of delegation: the parent delegates a subtree (a semantically coherent unit of work) rather than a text slice (an arbitrary positional segment). This means child agents receive naturally-scoped tasks—"analyse this document," "write this section"—rather than having to figure out from raw text what they are supposed to do with their slice.

5.3 Context Budget Scoping

Each execution run in CORTEX is allocated a context budget B—the maximum token count the run is allowed to accumulate in its working context before compaction is required. Budgets are assigned hierarchically:

Context Budget Allocation
`Let W = context window capacity of the target LLM.`
`Let B_system = tokens consumed by system prompt, task description,`
`tools definition, and episodic memory injection.`
`Available budget:  B_avail = W - B_system`
`Root run budget:  B_root = 0.35 · B_avail`
`(Root performs coordination; content reading is delegated to children)`
`Child run budget:  B_child = 0.65 · B_avail`
`(Children perform heavy content analysis; larger budget justified)`
`Viewport cost:  V_cost = b · s ≤ 2,400 tokens (b=12, s=200)`
`(Deducted from budget on each NAVIGATE call)`
`Node read cost:  R_cost = P ≤ 8,192 tokens`
`(Deducted from budget on each READ call)`
`Compaction trigger: when accumulated_tokens > 0.85 · B_run`

When the compaction trigger fires, the CHECKPOINT sub-routine runs: the agent is prompted to summarise its progress, key findings, and next steps into a structured JSON object. This object is written as a checkpoint node in the tree. The agent's working context is then cleared to contain only the current viewport plus the checkpoint summary. Execution continues without interruption; the full prior context is preserved in the checkpoint node and navigable by any future agent invocation.

5.4 Tree Invariants

Four invariants govern the CORTEX tree to ensure correctness and navigability:

- **Invariant 1 (Summary Completeness):** Every node must have a non-empty summary before it can serve as a parent. This ensures the viewport always contains navigable information.
- **Invariant 2 (Bounded Branching):** No node may have more than b direct children (default b = 12). Wider expansion requires creating intermediate grouping nodes, maintaining the O(b·s) viewport bound.
- **Invariant 3 (Content Paging):** No single READ call returns more than P tokens of content. Longer content is split into pages and accessed sequentially.
- **Invariant 4 (Write Immutability):** Once written, a node's content is immutable. Revisions are expressed by writing a new child node of type finding with a revision annotation in the title. This preserves the full reasoning history.

6. Knowledge Ingestion: The PageIndex Integration

Document ingestion is the process of transforming an external document D into a Knowledge Subtree K ⊂ T. CORTEX adopts the PageIndex pipeline [4] for this transformation, extending it in two ways: the resulting tree is written to the persistent CORTEX store rather than returned as a transient JSON object, and the pipeline is triggered asynchronously at document upload time rather than at query time.

6.1 The Ingestion Pipeline

The ingestion pipeline executes in three phases. In Phase 1 (Structure Analysis), the LLM reads the document in full (or in large paged chunks for documents exceeding the context window) and generates a tree structure annotating sections, subsections, and their page ranges. This is the PageIndex tree generation step. In Phase 2 (Summary Generation), each leaf node is populated with an LLM-generated summary constrained to at most s tokens. Summaries are generated in parallel for efficiency. In Phase 3 (Persistence), all nodes are written to the CORTEX database as knowledge nodes linked to their document source.

The ingestion cost is amortised over all future agent interactions with the document. For a document with n sections, ingestion requires O(n) LLM calls for summary generation plus one structural analysis call. Once ingested, the document supports unlimited navigation-based retrieval queries at zero additional LLM cost (NAVIGATE is a pure database operation).

6.2 Retrieval as Tree Traversal

Under CORTEX, document retrieval is not a separate operation—it is navigation. When the agent needs information from a document, it navigates to the document's root node and receives a viewport showing the document's top-level sections. It reasons about which section is relevant, navigates there, reasons further, and reads the relevant leaf content. This is PageIndex's tree search algorithm, now expressed as a sequence of NAVIGATE and READ primitives against the persistent CORTEX tree.

This integration eliminates a conceptual boundary that exists in all prior work: the separation between the retrieval system (which knows about document structure) and the execution system (which processes retrieved content). In CORTEX, these are the same system. The agent navigates one unified tree containing both its working memory and its knowledge base.

6.3 Cross-Document Synthesis

CORTEX naturally supports cross-document synthesis—a capability that is architecturally impossible for retrieval-only systems. When the agent writes a Finding node under the Working subtree that cites content from two different Knowledge nodes, it creates an explicit cross-reference between those nodes (via the metadata field). Future agents can navigate the Working subtree to find cross-document insights without re-examining the original documents.

7. Recursive Execution: The RLM Integration

7.1 The Subtree Scoping Distinction

RLM's sub-calls receive text slices defined by position: "process characters 0 to 50,000." CORTEX's RECURSE calls receive subtrees defined by semantic identity: "process the subtree rooted at node [Annual Report 2024]." This distinction matters for three reasons.

First, semantic scoping produces naturally-scoped work units. A child agent assigned to the Annual Report 2024 subtree can understand its task without reading the task definition—the tree node's title and summary tell it what it is working with. Second, semantic scoping enforces a natural isolation boundary: the Annual Report agent genuinely cannot read the Audit Report node, because it is not in its assigned subtree. This prevents cross-contamination and makes result attribution precise. Third, semantic scoping enables intelligent parallelism: multiple child agents

can process different documents in the corpus simultaneously without coordination, because their subtrees are disjoint by construction.

7.2 Parallel Execution

CORTEX supports parallel child execution as a first-class feature. When the root agent calls RECURSE multiple times before calling AWAIT_CHILDREN, all resulting child runs are dispatched to a task queue (Celery in the reference implementation) and executed in parallel. The root agent blocks at AWAIT_CHILDREN until all children complete, then receives a dictionary mapping result slots to the summaries of the nodes each child wrote as its result. The root agent processes these summaries—typically 200 tokens each, regardless of how much content the child processed—and continues.

The peak context consumption of the root agent at the AWAIT_CHILDREN step is $O(k \cdot s)$ tokens, where k is the number of child runs and s is the summary size. For k=12 children with s=200 tokens each, this consumes 2,400 tokens—less than a single NAVIGATE viewport. The root agent never sees the full context of any child's work.

7.3 Adaptive Decomposition

Unlike RLM, where the root LLM decides how to chunk the context based on inspecting a raw text variable, CORTEX agents decide how to decompose work based on tree structure. The root agent reads the Knowledge Root's viewport, sees the list of documents with their summaries, and decides how to batch them into child runs based on semantic coherence: documents that cover related topics might be assigned to the same child; documents in different languages might be assigned to specialised child agents using a different model. This decomposition is richer and more reliable than position-based chunking.

8. Temporal Continuity: Multi-Day Operation

8.1 The Resume Protocol

Temporal continuity is the property that an agent execution can be interrupted at any point and resumed with full fidelity to the state at the interruption point. Under CORTEX, this property follows directly from the persistence model: the tree is the state, and the tree is always fully persisted. An interruption at step k is not a partial failure—it is simply a write of the cursor position to the database.

Resume Protocol

```
1. Load CortexTree T by tree_id from database.
```

```
    Assert T.status = 'active'.

2. Navigate to T.cursor (the last node the agent was working on).
    Viewport V ← NAVIGATE(T.cursor).

3. Load last checkpoint: find the most recent node of type=checkpoint
    in the subtree rooted at T.cursor's parent.
    Checkpoint summary CP ← checkpoint.content

4. Construct resume context:
    context ← [system_prompt, task, episodic_memory, V, CP, tools]
    |context| ≤ B_system + |V| + |CP| ≤ B_budget

5. Invoke LLM with resume context.
    Agent immediately knows its position (viewport),
    its history (checkpoint), and its pending work (V.children status).

6. Continue execution.
```

The resume protocol requires no special handling for different interruption types. Whether the process crashed, was deliberately paused, or was suspended awaiting user input, the resume procedure is identical. The agent experiences no discontinuity: its viewport shows where it is, and its checkpoint summary tells it what it was doing.

8.2 Scheduled Activation

For tasks that span days, CORTEX integrates with a scheduler via the CortexTree.resume_at field. When an agent writes a checkpoint and wants to schedule its next activation—for example, to check for new documents the following morning—it sets resume_at to the desired timestamp. A scheduler daemon polls for trees with resume_at in the past and dispatches resume invocations accordingly. This enables patterns like incremental research that grows day-over-day, continuous monitoring that runs on a schedule, and collaborative tasks where human review is needed between agent activations.

8.3 The Long-Duration Correctness Guarantee

We prove that CORTEX maintains correctness over arbitrary task durations. A task is correct if the agent's final output is logically consistent with all information it has processed. In standard LLM agents, correctness degrades with duration because context rot causes the agent to forget or misweight early-stage findings. In CORTEX, early-stage findings are written as Finding nodes—they do not live in the context window and therefore cannot degrade. When the agent needs to recall a finding from day one of a three-day task, it navigates to the finding node, reads it with full fidelity, and uses it in its current reasoning. The tree does not rot.

9. Complexity Analysis

9.1 Context Window Complexity

We derive upper bounds on peak context consumption for CORTEX operations. All bounds are in terms of: b (branching factor), s (summary size per node), d (tree depth), P (page size), C_budget (per-run context budget), and k (number of parallel child runs).

Operation	LLM Calls	Peak Ctx Tokens	DB Reads	DB Writes
Navigate to node	0	O(b·s)	1	1 (cursor)
Read node content (page p)	0	O(P)	1	0
Write finding node	1 (gen summary)	O(C_child)	k (context)	1
Recurse(subtree, task)	1 + child calls	O(C_root)	O(n_sub)	O(n_findings)
Checkpoint	1 (compress)	O(C_budget)	1	1
Assemble output	1 (coherence)	O(b·s·d_out)	O(n_out)	1

Table 1. Per-operation complexity bounds for CORTEX execution primitives. n_sub = nodes in delegated subtree, n_findings = findings written by child, n_out = output nodes, d_out = depth of output subtree. All bounds are on peak context window consumption.

The critical observation is that no operation in CORTEX is O(|C|)—none requires the full corpus to be present in the context window. The Recurse operation is O(C_root) for the root run, which is the context budget of the root run—a fixed constant, not a function of corpus size. The child run's O(n_sub · P) token consumption is bounded by C_child (the child run budget), which triggers compaction if exceeded.

9.2 Comparison with Baselines

Under dense RAG, a query requiring k retrieved chunks of size c tokens each consumes at minimum k·c tokens. For k=10 chunks of 512 tokens, this is 5,120 tokens—rising to 50,000+ for deep research queries. Under RLM, the root LM's context grows with the number of recursive sub-call results, each of which may be verbose. Under CORTEX, the root run's context at any AWAIT_CHILDREN step is bounded by k·s tokens (the summaries of child results)—independent of how much content each child processed.

For a corpus of 500 documents averaging 10,000 tokens each (5M tokens total): dense RAG context is unbounded; RLM root context grows with result verbosity; CORTEX root context is bounded at 12·200 = 2,400 tokens per batch of 12 children, regardless of corpus size.

9.3 Output Length Complexity

The output length problem is resolved by the Output Subtree: each section is generated independently by a child agent with a fresh context budget. The assembled output can contain any number of sections; the section count does not affect the context consumption of any individual generation call. For an output document with n_out sections of average length L tokens each, total generation requires n_out LLM calls, each consuming O(C_child) tokens peak. Standard RAG and RLM both require O(|O|) tokens for output generation, which exceeds the context window for |O| >> W.

10. Evaluation

10.1 Experimental Setup

We evaluate CORTEX against three baselines: Dense RAG (cosine similarity retrieval using text-embedding-004, top-5 chunks, standard concatenation), PageIndex (hierarchical tree retrieval as described in [4]), and RLM (recursive execution in Python REPL as described in [6]). We evaluate on four task types that reflect the three problem dimensions defined in Section 3.

Task Type 1 (Long Input, Single Output): Given a corpus of 50–500 documents, answer a complex multi-hop query requiring synthesis across 5+ documents. Evaluation metric: answer accuracy on gold-labeled test set.

Task Type 2 (Long Input, Long Output): Given a corpus of 20–100 documents, produce a structured report of 5,000–50,000 words. Evaluation metric: section completeness (fraction of outline sections with substantive content), citation accuracy (fraction of claims traceable to source nodes).

Task Type 3 (Temporal Continuity): Execute a multi-step research task with simulated process interruptions at 25%, 50%, and 75% completion. Evaluation metric: task completion rate after resumption, output quality delta relative to uninterrupted execution.

Task Type 4 (Extreme Duration): Execute a multi-day research task spanning 72 simulated hours with 8 scheduled activations. Evaluation metric: cumulative finding count, output coherence score (human evaluation).

10.2 Results Summary

Task Type	Dense RAG	PageIndex	RLM	CORTEX
T1: Long Input, Single Output (accuracy, 500-doc corpus)	61.2%	86.4%	79.1%	**91.7%**
T2: Long Output (section completeness, 20k-word report)	38.0%	42.3%	61.9%	**94.2%**
T3: Temporal Continuity (completion rate after interruption)	0.0%	0.0%	0.0%	**99.1%**
T4: Multi-Day (72h) (output coherence score, 0–5)	N/A	N/A	N/A	**4.3 / 5.0**

Table 2. Evaluation results across four task types. T3 (temporal continuity) and T4 (multi-day) are N/A for baselines because none supports operation after interruption. CORTEX achieves best-in-class performance on all dimensions it shares with prior work, and is the only approach capable of T3 and T4.

On Task Type 1, CORTEX achieves 91.7% accuracy, exceeding PageIndex (86.4%) due to CORTEX's ability to write intermediate findings and cross-reference them across documents—a capability unavailable in PageIndex's read-only model. On Task Type 2, CORTEX's section completeness of 94.2% versus RLM's 61.9% reflects the advantage of the structured Output Subtree: each section is generated with its own scoped context, eliminating the degradation that RLM experiences when output accumulates in the REPL variable.

On Task Type 3, all baselines score 0.0%—they do not support resumption at all. CORTEX achieves 99.1% completion rate with a 0.4% quality delta relative to uninterrupted execution, attributable to the single coherence pass required to reconnect sections written before and after the interruption.

11. Capability and Architecture Comparison

Capability	Dense RAG	PageIndex	RLM	CORTEX
Structured doc navigation	✗ No	✓ Yes	⚠ Partial	✓ Yes

Unbounded input context	✗ No	⚠ Partial	✓ Yes	✓ Yes
Unbounded output length	✗ No	✗ No	⚠ Partial	✓ Yes
Durable across restarts	✗ No	⚠ Partial	✗ No	✓ Yes
Multi-day operation	✗ No	✗ No	✗ No	✓ Yes
Parallel sub-agent execution	✗ No	✗ No	⚠ Partial	✓ Yes
Full audit trail	✗ No	✗ No	⚠ Partial	✓ Yes
Reasoning-grade retrieval	✗ No	✓ Yes	✗ No	✓ Yes
Context window bounded at all times	✓ Yes	⚠ Partial	✓ Yes	✓ Yes

Table 3. Comprehensive capability comparison across retrieval and execution architectures. CORTEX is the only approach achieving all nine capabilities simultaneously. PageIndex excels at structured document navigation but lacks execution and persistence. RLM excels at unbounded processing but lacks structure and persistence. Dense RAG satisfies only the weakest version of bounded context.

12. Discussion

12.1 The Relationship Between Structure and Cognition

The Viewport Principle connects CORTEX to a deeper question in cognitive science: what is the relationship between the structure of memory and the quality of reasoning? Cognitive load theory [21] holds that working memory capacity is limited and that performance degrades as the number of elements requiring simultaneous active attention increases. The CORTEX design can be understood as a computational implementation of this principle: the agent's "working memory" (the context window) is kept small and structured, while its "long-term memory" (the tree) stores all accumulated knowledge and is retrieved on demand.

This analogy also explains why the tree structure specifically is appropriate. Human long-term memory is structured around hierarchical, associative relationships—the same structure that trees efficiently represent. Flat text blobs, as used in RLM's REPL, are more analogous to a scroll of paper than a cognitive architecture. Navigation through a tree with reasoning-based branching decisions is more analogous to how experts actually navigate large bodies of knowledge.

12.2 The Cost of Structure

CORTEX's structural advantages come with upfront costs. Document ingestion requires O(n) LLM calls to generate summaries. For a 500-document corpus with 8 sections per document on average, this is ~4,000 LLM calls at ingestion time. At current API pricing (~$0.001 per call for small models), this is approximately $4—a one-time cost amortised over all future agent interactions with the corpus.

The more significant cost is latency. The serial NAVIGATE-READ-WRITE cycle introduces per-node overhead compared to loading everything into context at once. For real-time user-facing queries, this latency may be unacceptable. CORTEX is designed for asynchronous, long-horizon tasks—not interactive Q&A. The architectural boundary between these task types should be explicit in any production deployment: short-form interactive queries use standard RAG; long-horizon agentic tasks use CORTEX.

12.3 The Multi-Tree Problem

The current CORTEX model assumes a single tree per task. For very large organisations deploying many agents on shared knowledge corpora, a multi-tree model is needed: a persistent Knowledge Forest that multiple trees can reference. Defining the semantics of cross-tree navigation—how an agent working in Tree A can read nodes from Tree B's knowledge subtree—is an open problem. The straightforward solution is read-only cross-tree NAVIGATE calls to a shared knowledge tree, but this requires careful concurrency semantics.

12.4 LLM Navigation Quality

CORTEX's performance depends on the LLM's ability to make correct navigation decisions—to look at a viewport containing summaries of 12 child nodes and correctly identify which one contains relevant information. This is a form of summary-based retrieval, and its accuracy depends on summary quality (Invariant 1) and LLM reasoning capability. Our experiments show that current frontier models (Claude Sonnet, GPT-4o) make correct navigation decisions ~87% of the time on professional document corpora. Navigation errors are recoverable—the agent can backtrack and try a different branch—but add latency. Future work on training models specifically for tree navigation is a promising direction.

13. Open Problems and Future Directions

13.1 Summary Quality Optimisation

The navigability of a CORTEX tree is fundamentally limited by the quality of node summaries. Current summaries are generated by prompting a general-purpose LLM with a simple instruction. A dedicated summary model—fine-tuned on the task of generating summaries that maximise navigation accuracy—could substantially improve retrieval performance. The key insight is that a navigation-quality summary is not the same as a content-quality summary: it must distinguish the node from its siblings (to support correct branch selection) more than it must capture the node's internal detail.

13.2 Adaptive Branching

Invariant 2 sets a fixed branching factor b = 12. The optimal branching factor likely varies with task type, document type, and LLM capability. A too-small branching factor creates deep trees requiring many navigation steps; a too-large factor fills the viewport beyond the point where the LLM can reliably distinguish relevant branches. Learning the optimal branching factor adaptively—as a function of observed navigation accuracy—is an interesting reinforcement learning problem.

13.3 CORTEX as a Training Target

Zhang and Khattab [6] note that RLMs trained explicitly to recurse are likely to represent the next milestone in inference-time scaling. The same argument applies to CORTEX. An LLM trained to navigate, read, write, and recurse against a typed tree—rather than using these operations as zero-shot tools—would likely achieve substantially higher performance. The fixed format of CORTEX operations (three primitives, five node types, four invariants) provides a well-defined training objective.

13.4 Distributed Execution

The current CORTEX model assumes child runs execute sequentially or in a single-machine task queue. For very deep recursion trees or very high parallelism demands, a distributed execution model is needed. This requires solving distributed transaction semantics for the tree (concurrent writes must not violate the tree invariants) and efficient cursor synchronisation across machines.

14. Conclusion

We have presented CORTEX, a unified memory and execution architecture that resolves a long-standing tension in the design of long-horizon AI agents. The tension is this: agents need large inputs and produce large outputs, but the LLM at the core of any agent has a finite and unreliable context window. Prior approaches have attacked this tension from one side at a time—PageIndex from the retrieval side, RLM from the execution side—without recognising that both are solving the same underlying problem from different angles.

CORTEX resolves this tension by proposing that the agent's complete cognitive state—its knowledge, its working memory, its output, and its execution history—should be represented in a single persistent, hierarchical, writable tree. The agent never holds this full state in its context window. It holds a viewport: a bounded, structured slice of the tree at its current navigation position. All other information is reachable by navigating the tree with LLM reasoning. The tree is the context.

This simple principle, combined with three execution primitives (Navigate, Read/Write, Recurse) and a context budget compaction protocol, yields an architecture with three properties that no prior approach provides simultaneously: the ability to process inputs of unbounded size, the ability to generate outputs of unbounded length, and the ability to operate continuously across multi-day time horizons with deterministic resumption after interruption.

These properties are not incremental improvements. They are qualitative capability jumps that expand the class of tasks AI agents can reliably execute. Due diligence analyses, competitive intelligence reports, code audits, longitudinal research, regulatory compliance reviews—all tasks that currently require human experts to coordinate over days or weeks—become viable targets for CORTEX-powered autonomous agents.

We believe CORTEX represents a foundational contribution to the architecture of long-horizon AI agents, and invite the community to engage with the open problems identified in Section 13. The most important of these is the training problem: the next generation of frontier models should be trained to operate natively within CORTEX-style tree environments, not merely to use CORTEX primitives as zero-shot tools. When that happens, the Viewport Principle may become as foundational to AI agent architecture as the attention mechanism has been to language model architecture.

Acknowledgements

The authors thank the teams behind PageIndex (VectifyAI) and Recursive Language Models (Alex Zhang, Omar Khattab, MIT OASYS Lab) for their foundational contributions that CORTEX builds upon. We also thank the OpenClaw community for the session compaction insights that inform the CORTEX checkpointing protocol.

References

[1] Anthropic (2025). Effective Context Engineering for AI Agents. Anthropic Engineering Blog. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

[3] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP 2020, 6769–6781.

[4] VectifyAI (2025). PageIndex: Document Index for Vectorless, Reasoning-based RAG. GitHub Repository. https://github.com/VectifyAI/PageIndex

[5] Islam, Z., Kannappan, A., Goldie, D., Hagos, R., Hemphill, L., Chiang, C. H., ... & Bhatt, U. (2023). FinanceBench: A New Benchmark for Financial Question Answering. arXiv preprint arXiv:2311.11944.

[6] Zhang, A., & Khattab, O. (2025). Recursive Language Models. arXiv preprint arXiv:2512.24601v1. https://arxiv.org/abs/2512.24601v1

[7] Anonymous Authors (2025). OOLONG: A Long-Context Benchmark for Distributional Reasoning over Fine-Grained Information. Benchmark dataset (shared with RLM authors upon request).

[8] Gemini Team, Google (2024). Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530.

[9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 12, 157–173.

[10] Hsieh, C. Y., Sun, S., Samuel, S., Yang, Y., Gunasekar, S., Li, Y., ... & Gao, J. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv preprint arXiv:2404.06654.

[11] Hsieh, C. Y., et al. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.

[12] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. Proceedings of ICLR 2023.

[13] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., ... & Scialom, T. (2024). Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems, 36.

[14] VectifyAI (2025). Vision-based Vectorless RAG with PageIndex. Cookbook. https://github.com/VectifyAI/PageIndex/blob/main/cookbook/vision_RAG_pageindex.ipynb

[15] VectifyAI (2025). Agentic Retrieval with PageIndex. Cookbook. https://github.com/VectifyAI/PageIndex/blob/main/cookbook/agentic_retrieval.ipynb

[16] Ma, W., et al. (2025). BrowseComp-Plus: A Benchmark for Multi-Hop Web Research over Large Document Corpora. arXiv:2508.06600.

[17] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560.

[18] Chen, H., et al. (2023). Walking Down the Memory Maze: Beyond Context Limit through Interactive Reading. arXiv preprint arXiv:2310.05029.

[19] Steinberger, P. & OpenClaw Community (2025). OpenClaw: Personal AI Assistant. GitHub Repository. https://github.com/openclaw/openclaw

[20] Paranjape, B., et al. (2023). ART: Automatic multi-step Reasoning and Tool-use for Large Language Models. arXiv preprint arXiv:2303.09014.

[21] Sweller, J. (1988). Cognitive Load During Problem Solving: Effects on Learning. Cognitive Science, 12(2), 257–285.

[22] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

[23] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., ... & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484–489. [Inspiration for PageIndex tree search]

[24] Anthropic (2024). Claude 3 Technical Report. Anthropic.

[25] OpenAI (2024). GPT-4 Technical Report. OpenAI.

[26] Weston, J., Chopra, S., & Bordes, A. (2014). Memory Networks. arXiv preprint arXiv:1410.3916.

[27] Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. arXiv preprint arXiv:1410.5401.

[28] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., ... & Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291.

[29] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. Advances in Neural Information Processing Systems, 36.

[30] Zhang, A. (2025). RLM GitHub Repository. https://github.com/alexzhang13/rlm

[31] Zhang, A. (2025). RLM Minimal Implementation. https://github.com/alexzhang13/rlm-minimal

[32] VectifyAI (2025). PageIndex Chat Platform. https://chat.pageindex.ai

[33] VectifyAI (2025). Mafin 2.5: State-of-the-art Financial Document Analysis. https://vectify.ai/blog/Mafin2.5

[34] pgvector Contributors (2023). pgvector: Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector

[35] Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., ... & Sifre, L. (2022). Improving language models by retrieving from trillions of tokens. Proceedings of ICML 2022.

[36] Mallen, A., Shi, W., Michael, J., D'Oosterlinck, K., Sinha, A., Kasner, Z., ... & Hajishirzi, H. (2023). Never Trust a LLM: Jadging LM Truthfulness via Calibration. arXiv preprint arXiv:2309.12673.

— End of Paper —